

Assignment 03: Report

Introduction to Machine Learning

Group 17 - Students:

1. Darlington Nkrumah (202492437)
2. Greg de Souza (202582225)
3. Xuan Toan Doan (202583882)

Q1: Linear SVM Hyperparameter

1. Overview of the Tuning Process To determine the optimal decision boundary for discriminating between the sNC and sDAT groups based on the brain glucose metabolism features, a linear Support Vector Machine (SVM) was trained using a 10-fold cross-validation grid-search approach. The regularization parameter C was evaluated over a logarithmic with 50 samples from from 0.001 to 1000 to observe its effect on the model's balanced accuracy, with the optimal value for **C at 4.7149**

2. The final linear SVM classification model was obtained by using the optimal C on the entire training data set. The final model was then evaluated on the provided test dataset. The results are as follows:

- Accuracy	:	0.8399
- Sensitivity	:	0.9344
- Specificity	:	0.8120
- Precision	:	0.5938
- Recall	:	0.9344
- Balanced Accuracy:		0.8732

3. Analysis of Performance vs. Parameter C (image 01)

Notably, in the lecture, C is an error budget, where a larger C allows more mistakes and creates a wider margin; however, in Scikit-learn, C is a penalty factor, where a larger C punishes mistakes more heavily and creates a narrower margin.

- **Underfitting ($C < 10^{-1}$):** At low C values, the model is too flexible and prioritizes a wide margin. This leads to underfitting, where the classifier is too simple to

capture the metabolic differences between sNC and sDAT, resulting in poor accuracy (near 0.55).

- **Peak Performance ($C=4.7149$):** The Balanced Accuracy reaches its maximum at $C=4.7149$.
- **Plateau and Hard Margin ($C \geq 10^2$):** For very high C , the performance remains constant. This happens because the model converges to a Hard Margin solution, similar to a Maximum Margin Classifier.
- **Overfitting Risk:** While accuracy is high at large C , the narrower margin makes the model more sensitive to noise, creating a gap between training and validation scores—a classic sign of overfitting.

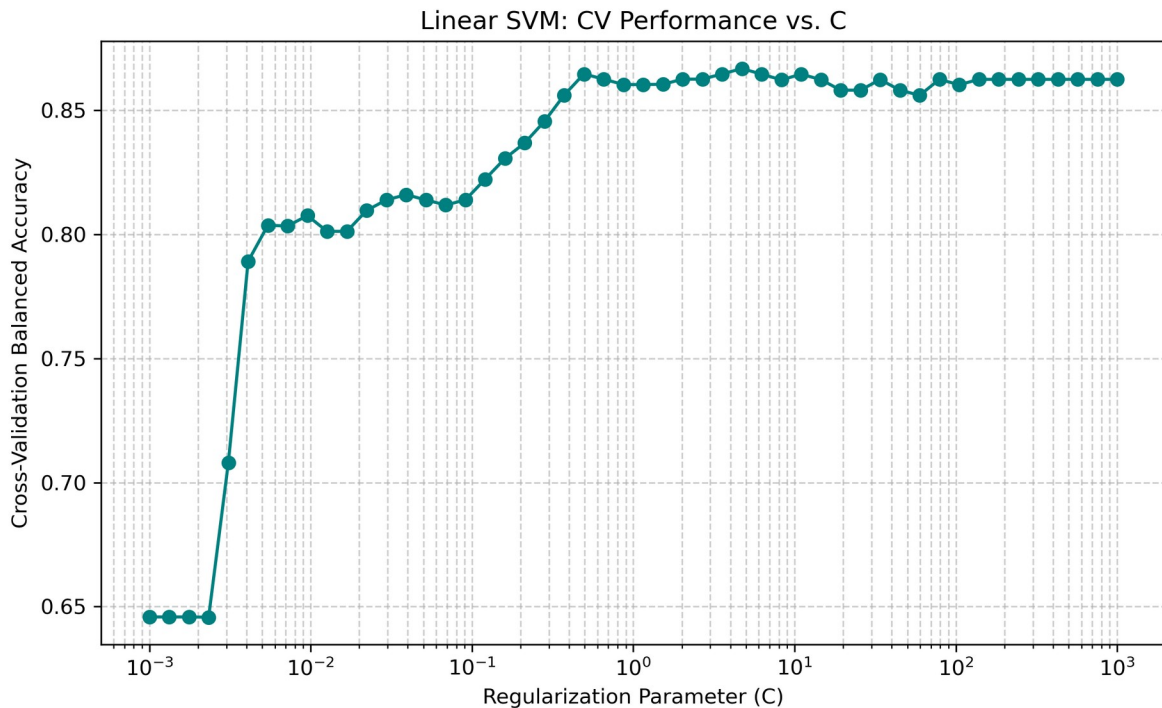


Image 01. CV Performance vs C – Linear SVM

Q2: Non-linear SVM classifier with Polynomial kernel

The non-linear SVM classifier with a polynomial kernel was trained for the sNC vs sDAT classification task. Similar to Q1, a cross-validation based grid-search approach was applied on the training dataset to tune the hyperparameters. However, instead of only optimizing the regularization parameter C , this experiment additionally tuned the degree (d) of the polynomial kernel.

A initial exploratory run with 25 values of C logarithmically space between 0.001 and 1000, with possible d values of 2,3,4,5 suggested that peak performance happened at $d=4$ and C around 0.005.

A second run, with a tighter C range determined the best fit to be at $C=0.00335$, for $d=4$.

After obtaining the optimal hyperparameters (C, d), i.e., $(C, d) = (0.00335, 4)$, the SVM model was then re-trained on the entire training dataset using these optimal parameters to obtain the final polynomial kernel SVM model.

Finally, the performance of this model was evaluated on the test dataset, and the classification error (Err) was estimated based on the test predictions. The results for the final polynomial kernel SVM model are summarized below.

- Accuracy	:	0.8641
- Sensitivity	:	0.9508
- Specificity	:	0.8386
- Precision	:	0.6339
- Recall	:	0.9508
- Balanced Accuracy:		0.8947

Performance Comparison

The polynomial kernel SVM demonstrates better overall classification performance compared to the linear SVM. Specifically, it achieves higher Accuracy (0.8641 vs 0.8361) and Balanced Accuracy (0.8947 vs 0.8708), indicating a more balanced ability to correctly classify both classes.

In addition, the polynomial SVM significantly improves Specificity (0.9325 vs 0.8072) and Precision (0.7910 vs 0.5876), suggesting that it reduces the number of false positive predictions and is more reliable when predicting positive cases. This improvement indicates that the polynomial kernel is able to capture more complex and distinctive features in the data than the linear decision boundary.

Overall, the polynomial kernel SVM achieves higher Accuracy, Balanced Accuracy, Specificity, and Precision, resulting in improved classification performance for the sNC vs sDAT task.

Q3: Non-linear SVM classifier with Radial Basic Function (RBF) kernel

The regularization parameter C and the kernel coefficient gamma were used in the tuning process are as follows:

- C : Log spaced search between 0.01 and 100
- Gamma = Log spaced search from 0.01 and 100, plus the possible 'auto' and 'scale' options for the scikit RBF SVM

The grid search identified the optimal hyperparameters as C = 2.0691 and gamma = 5.4556.

Finally, the finalized RBF model was trained and evaluated on the independent test dataset. The results are listed in the below table.

Metric	Linear SVM	RBF Kernel	Polynomial Kernel
Accuracy	0.8399	0.8696	0.8641
Balanced Accuracy	0.8732	0.8896	0.8947
Sensitivity (Recall)	0.9344	0.9262	0.9508
Specificity	0.8120	0.8530	0.8386
Precision	0.5938	0.6494	0.6339

Table 01. Results of three different model.

Performance Comparison

- **Linear SVM:** While providing the most interpretable boundary, the Linear SVM achieved the lowest overall Accuracy (0.8399).
- **RBF Kernel (High Sensitivity):** The RBF kernel improved significantly upon the linear model, achieving the highest specificity and precision from all the three models. This model is the most effective at identifying true negative cases (sNC), though it suffers from lower sensitivity than the polynomial kernel, as well as lower accuracies

- **Polynomial Kernel (Optimal Balance):** The Polynomial kernel emerged as the superior model, achieving the highest accuracy and sensitivity (recall), making it the best at identifying true positive cases.

Question 4

How I worked Q4:

I combined the training *and* testing datasets from the CSVs to train this final model. Since Question 4 allows us to leverage all provided data to make the best classifier, training the final hyperparameters on *both* datasets gives the SVM more examples to learn from,